

Paradigmas y Lenguajes de Programación 2026

Guía de Estudio — Tema 03

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1	Guía de Estudio — Tema 03	2
2	Introducción a la Programación Funcional con TypeScript	2
2.1	Índice	2
2.2	1. Objetivos de aprendizaje	3
2.3	2. Conceptos previos necesarios	3
2.4	3. Fundamentos del paradigma funcional	3
2.4.1	3.1 Cómputo sin estado	3
2.4.2	3.2 Funciones puras	4
2.4.3	3.3 Inmutabilidad	5
2.4.4	3.4 Recursión como control de flujo	6
2.5	4. Funciones de orden superior y clausuras	7
2.5.1	4.1 Funciones como valores de primera clase	7
2.5.2	4.2 map, filter, reduce	8
2.5.3	4.3 Clausuras y ámbito léxico	9
2.6	5. Composición, currificación y aplicación parcial	10
2.6.1	5.1 Composición de funciones	10
2.6.2	5.2 Currificación	11
2.7	6. Manejo de efectos y evaluación perezosa	12
2.7.1	6.1 Aislar los efectos	12
2.7.2	6.2 Evaluación perezosa con generadores	13
2.8	7. Introducción a mónadas	14
2.8.1	7.1 El problema que resuelven	14
2.8.2	7.2 Option / Maybe — mónada de opcionalidad	15
2.8.3	7.3 Either — mónada de error explícito	15
2.8.4	7.4 Promise como mónada	16
2.9	8. Ejemplos trabajados paso a paso	17
2.9.1	Ejemplo 1: Pipeline de procesamiento de encuesta	17
2.9.2	Ejemplo 2: Mini-librería de transformaciones funcionales	18
2.10	9. Puntos clave del tema	20
2.11	10. Autoevaluación	20
2.11.1	Bloque A — Preguntas conceptuales	20
2.11.2	Bloque B — Ejercicios de código	20

2.12	11. Glosario	22
2.13	12. Referencias	23

1 Guía de Estudio — Tema 03

2 Introducción a la Programación Funcional con TypeScript

Agente: Dra. Sofia (study-guide-writer) **Fecha de generación:** 2026-03-13 **Basada en:** `diseño.md` (aprobado) · `minuta.md` · `filminas.md` **Fuentes integradas:**
 - Gabbrielli, M. & Martini, S. (2023). *Programming Languages: Principles and Paradigms*, Cap. 11. - Sebesta, R. (2018). *Concepts of Programming Languages*, 12th ed., Cap. 15. - Material de cátedra: *Introducción a la Programación Funcional* — UNTDF, 2025. **Materia:** Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI

2.1 Índice

1. [Objetivos de aprendizaje](#)
 2. [Conceptos previos necesarios](#)
 3. [Fundamentos del paradigma funcional](#)
 - 3.1 Cómputo sin estado
 - 3.2 Funciones puras
 - 3.3 Inmutabilidad
 - 3.4 Recursión como control de flujo
 4. [Funciones de orden superior y clausuras](#)
 - 4.1 Funciones como valores de primera clase
 - 4.2 `map`, `filter`, `reduce`
 - 4.3 Clausuras y ámbito léxico
 5. [Composición, currificación y aplicación parcial](#)
 6. [Manejo de efectos y evaluación perezosa](#)
 7. [Introducción a mónadas](#)
 8. [Ejemplos trabajados paso a paso](#)
 9. [Puntos clave del tema](#)
 10. [Autoevaluación](#)
 11. [Glosario](#)
 12. [Referencias](#)
-

2.2 1. Objetivos de aprendizaje

Al finalizar el estudio de este tema, debés poder:

1. **Explicar** los fundamentos del paradigma funcional: funciones puras, inmutabilidad, recursión.
2. **Distinguir** una función pura de una impura y justificar por qué esa distinción importa.
3. **Implementar** funciones de orden superior (`map`, `filter`, `reduce`) y usarlas para transformar datos.
4. **Construir** pipelines de transformación usando composición (`pipe/compose`) y currificación.
5. **Identificar** el patrón mónada en código TypeScript cotidiano (`Promise`, `Option`, `Either`).
6. **Contrastar** el paradigma funcional con el imperativo en términos de modelo de cómputo, testabilidad y composabilidad.

2.3 2. Conceptos previos necesarios

Antes de comenzar, asegurate de dominar:

Concepto	Dónde lo viste
Funciones en TypeScript: declaración, arrow functions, tipos	Tema 01
Arreglos y métodos nativos (<code>Array.prototype.*</code>)	Tema 01
<code>var</code> , <code>let</code> , <code>const</code> y alcance de variables	Tema 01
Genéricos básicos en TypeScript (<code><T></code>)	Tema 01
Diferencia entre paradigma imperativo y declarativo	Tema 01

Si alguno de estos conceptos no está claro, revisá las filminas del Tema 01 antes de seguir.

2.4 3. Fundamentos del paradigma funcional

2.4.1 3.1 Cómputo sin estado

El paradigma imperativo concibe el cómputo como una secuencia de **instrucciones que modifican la memoria**. La Máquina de von Neumann es su modelo físico: un procesador que lee y escribe celdas de memoria paso a paso.

El paradigma funcional tiene un modelo radicalmente diferente: el cómputo avanza **evaluando expresiones**, no modificando estado. La base teórica no es la Máquina de Turing sino el **-cálculo** (Alonzo Church, 1930s):

“In pure functional languages, there is neither a state nor a modifiable variable. The computation proceeds — at least in principle — by rewriting expressions.”

— Gabbrielli & Martini, Cap. 11

Esta distinción tiene consecuencias profundas:

Imperativo	Funcional
Cómputo = modificar memoria	Cómputo = evaluar expresiones
Estado mutable (variables)	Valores inmutables
Bucles (for , while)	Recursión + funciones de orden superior
Efectos secundarios frecuentes	Efectos aislados en los bordes
Modelo: von Neumann	Modelo: -cálculo

Dato histórico: El paradigma funcional es tan antiguo como el imperativo. LISP (1958) fue el primer lenguaje inspirado en funciones matemáticas puras. Le siguieron Scheme, ML, Miranda y Haskell (lenguaje funcional puro moderno). TypeScript, Python y Java adoptaron ideas funcionales de forma no exclusiva.

2.4.2 3.2 Funciones puras

Una **función pura** cumple dos condiciones:

1. **Determinismo:** para los mismos argumentos, siempre devuelve el mismo resultado.
2. **Sin efectos secundarios:** no modifica nada fuera de su scope (no muta variables globales, no hace I/O, no lanza excepciones como efecto lateral).

```
// PURA - mismo input siempre produce mismo output, sin efectos
const doble = (n: number): number => n * 2;
const sumar = (a: number, b: number): number => a + b;

// IMPURA - depende de estado externo (puede cambiar entre llamadas)
let factor = 2;
const dobleConFactor = (n: number): number => n * factor;

// IMPURA - modifica estado externo
const historial: number[] = [];
```

```
const agregarYDoble = (n: number): number => {
  historial.push(n); // ↗ efecto secundario: modifica variable global
  return n * 2;
};

// IMPURA - depende del tiempo/entorno (no determinista)
const ahora = (): number => Date.now();
```

¿Por qué las funciones puras son tan valiosas?

- **Testeables:** no requieren mocks, stubs ni setup de entorno.
- **Componibles:** se pueden encadenar sin efectos inesperados.
- **Razonables:** el nombre + la firma dicen todo; no hay estado oculto que leer.
- **Paralelizables:** sin estado compartido, no hay condiciones de carrera.
- **Memoizables:** el resultado puede cachearse de forma segura (**memo**).

“Purely functional programs are easier to understand, both during and after development, largely because the meanings of expressions are independent of their context.”

— Robert Backus, 1977 ACM Turing Award Lecture (citado en Sebesta, Cap. 15)

2.4.3 3.3 Inmutabilidad

En el paradigma funcional **no se muta** un dato — se crea una nueva versión con las modificaciones deseadas.

```
// Imperativo - muta el array original (efecto secundario)
const agregarMutable = (arr: number[], item: number): void => {
  arr.push(item); // ↗ arr queda modificado para quien lo pasó
};

// Funcional - retorna un nuevo array, el original queda intacto
const agregarInmutable = (arr: readonly number[], item: number): readonly number[] =>
  [...arr, item];
```

Herramientas de inmutabilidad en TypeScript:

Herramienta	Qué garantiza
<code>const</code>	La <i>referencia</i> no puede reasignarse (no el contenido)
<code>readonly T[]</code>	El array no puede ser mutado a través de este tipo

Herramienta	Qué garantiza
<code>as const</code>	Convierte literales en tipos inmutables profundos
<code>Spread [...arr]</code>	Crea una copia superficial del array
<code>Object.freeze()</code>	Congela un objeto (inmutabilidad en runtime)

Métodos que NO mutan (funcionales): `map`, `filter`, `reduce`, `slice`, `concat`, `flat`, `flatMap`

Métodos que SÍ mutan (cuidado): `push`, `pop`, `splice`, `sort`, `reverse`, `fill`

```
// String de precaución: sort() muta el array original
const nums = [3, 1, 2];
nums.sort(); // ← muta nums!

// Versión segura:
const numsSorted = [...nums].sort(); // ← copia primero, luego ordena
```

2.4.4 3.4 Recursión como control de flujo

Sin variables mutables, los bucles tradicionales pierden sentido (un `for` necesita una variable `i` que se modifica). En el paradigma funcional, el mecanismo equivalente es la **recursión**:

“Without assignment, iteration becomes less important. In the stateless computation model, (unbounded) iteration disappears, recursion remains, becoming the fundamental construct for sequence control.”

— Gabbrielli & Martini, Cap. 11

```
// Imperativo - usa estado mutable (total)
const sumaImperativa = (nums: number[]): number => {
  let total = 0;
  for (const n of nums) total += n;
  return total;
};

// Funcional - sin estado mutable
const sumaRecursiva = (nums: number[]): number =>
  nums.length === 0
    ? 0
    : nums[0] + sumaRecursiva(nums.slice(1));
```

```
// Funcional con reduce (preferido en JS/TS por eficiencia)
const suma = (nums: number[]): number =>
  nums.reduce((acc, n) => acc + n, 0);
```

Recursión de cola (Tail Call Optimization — TCO):

Una función es **recursiva de cola** si la llamada recursiva es la última operación antes de retornar. En lenguajes funcionales puros (Haskell, Scheme) esto se optimiza automáticamente. En JavaScript/TypeScript, el motor V8 **no implementa TCO** de forma confiable, por lo que para arreglos grandes es más seguro usar `reduce`.

```
// Tail-recursive factorial (no optimizado en V8)
const factorialTail = (n: number, acc = 1): number =>
  n <= 1 ? acc : factorialTail(n - 1, n * acc);

// Versión con reduce (práctica y segura en JS/TS)
const factorial = (n: number): number =>
  Array.from({ length: n }, (_, i) => i + 1).reduce((acc, x) => acc * x, 1);
```

2.5 4. Funciones de orden superior y clausuras

2.5.1 4.1 Funciones como valores de primera clase

En TypeScript (y JavaScript), las funciones son **valores de primera clase**: pueden guardarse en variables, pasarse como argumentos y retornarse como resultado. Esta característica es el fundamento de toda la programación funcional.

```
// 1. Guardadas en variables
const saludar = (nombre: string): string => `Hola, ${nombre}!`;

// 2. Pasadas como argumento
const aplicar = (fn: (x: number) => number, valor: number): number => fn(valor);
console.log(aplicar(x => x ** 2, 5)); // 25

// 3. Retornadas como resultado - "función generadora"
const multiplicadorPor = (factor: number) => (n: number) => n * factor;
const triple = multiplicadorPor(3);
const cuadruple = multiplicadorPor(4);

console.log(triple(5)); // 15
```

```
console.log(cuadruple(5)); // 20
```

Esto es lo que en ML/Haskell se describe como funciones como “**expressible values**” (Gabbrielli & Martini, §11.1.1): una función puede ser el resultado de evaluar cualquier expresión.

2.5.2 4.2 map, filter, reduce

Son las tres funciones de orden superior más importantes. Implementarlas desde cero ayuda a entender cómo la recursión y las HOF se relacionan.

```
// Implementación desde cero con reduce
const myMap = <A, B>(fn: (a: A) => B, arr: readonly A[]): B[] =>
  arr.reduce<B[]>((acc, x) => [...acc, fn(x)], []);

// Equivalente nativo
const cuadrados = [1, 2, 3, 4, 5].map(n => n ** 2);
// [1, 4, 9, 16, 25]
```

2.5.2.1 map — transformar cada elemento

```
// Implementación desde cero con reduce
const myFilter = <A>(pred: (a: A) => boolean, arr: readonly A[]): A[] =>
  arr.reduce<A[]>((acc, x) => pred(x) ? [...acc, x] : acc, []);

// Equivalente nativo
const pares = [1, 2, 3, 4, 5, 6].filter(n => n % 2 === 0);
// [2, 4, 6]
```

2.5.2.2 filter — seleccionar por predicado

2.5.2.3 reduce — colapsar en un valor reduce (también llamado fold en Haskell/ML) es la función más general: map y filter son casos especiales de reduce.

```
// reduce para sumar
const suma = [1, 2, 3, 4, 5].reduce((acc, n) => acc + n, 0); // 15

// reduce para construir un objeto (frecuencias)
```



```
const frecuencias = (palabras: readonly string[]): Record<string, number> =>
  palabras.reduce<Record<string, number>>(
    (acc, palabra) => ({ ...acc, [palabra]: (acc[palabra] ?? 0) + 1 }),
    {}
  );

frecuencias(["hola", "mundo", "hola", "TS"]);
// { hola: 2, mundo: 1, TS: 1 }
```

Composición de los tres:

```
// Pipeline funcional: filtrar + transformar + reducir
const resultado = [1, 2, 3, 4, 5, 6]
  .filter(n => n % 2 === 0)           // [2, 4, 6]
  .map(n => n ** 2)                   // [4, 16, 36]
  .reduce((acc, n) => acc + n, 0); // 56
```

2.5.3 4.3 Clausuras y ámbito léxico

Una **clausura** (*closure*) es una función que captura las variables de su entorno de definición, incluso después de que ese entorno ya no existe en el stack de llamadas.

```
const crearMultiplicador = (factor: number) => {
  // `factor` es capturado por la clausura
  return (n: number): number => n * factor;
};

const doble = crearMultiplicador(2);
const triple = crearMultiplicador(3);

doble(5); // 10
triple(5); // 15
// `factor` sigue disponible aunque crearMultiplicador ya retornó
```

Ámbito léxico: la función captura el entorno donde fue **definida**, no donde fue **llamada**. Esto es fundamental para entender el comportamiento de las clausuras.

```
const x = 10;
const f = () => x; // captura x = 10 del entorno de definición
```

```
const crearFn = () => {
  const x = 99; // x local, distinta de la x externa
  return f;    // f sigue capturando x = 10
};

crearFn()(); // 10, no 99 - ámbito léxico, no dinámico
```

El ejemplo de `crearContador` con estado mutable interno (visto en clase) **no es puramente funcional** — tiene efectos internos. En funcional puro, el estado se pasa como argumento explícito (*state passing style*).

2.6 5. Composición, currificación y aplicación parcial

2.6.1 5.1 Composición de funciones

“*comp f g x = f(g(x))*”

— Gabbrielli & Martini, §11.1.1 (notación ML)

Componer funciones es la operación central del paradigma funcional: permite construir transformaciones complejas a partir de piezas pequeñas y simples.

```
// compose: aplica g primero, luego f (orden matemático: f  g)
const compose = <A, B, C>(  
  f: (b: B) => C,  
  g: (a: A) => B  
) => (a: A): C => f(g(a));

// pipe: aplica de izquierda a derecha (más natural para leer código)
const pipe = <A, B, C>(  
  g: (a: A) => B,  
  f: (b: B) => C  
) => (a: A): C => f(g(a));

// Ejemplo práctico
const limpiar = (s: string): string => s.trim();
const mayus   = (s: string): string => s.toUpperCase();
const excl    = (s: string): string => `${s}!`;

const gritar = pipe(pipe(limpiar, mayus), excl);
```

```
gritar(" hola mundo "); // "HOLA MUNDO!"
```

Ventaja de la composición: cada función hace *una sola cosa bien*. La complejidad emerge de combinar piezas simples, no de escribir funciones monolíticas.

2.6.2 5.2 Currificación

La **currificación** (por Haskell Curry, matemático) transforma una función de múltiples argumentos en una cadena de funciones de un solo argumento:

$$f(a, b) \rightarrow f(a)(b)$$

En ML/Haskell, todas las funciones son currificadas por defecto. En TypeScript es una elección de diseño:

Paso 1: Función tradicional de dos argumentos

```
const add = (a: number, b: number): number => a + b;
add(3, 4); // 7
```

Paso 2: Explícita — función que retorna función

```
const addExplicita = (a: number) => {
  return (b: number): number => a + b;
};
addExplicita(3)(4); // 7 - primero aplicas a 3, obtienes función de 1 arg, luego aplicas a 4
```

Paso 3: Compacta — notación arrow moderna

```
const addCurried = (a: number) => (b: number): number => a + b;
addCurried(3)(4); // 7

// Aquí está la clave: addCurried(5) NO devuelve un número
// sino UNA FUNCIÓN que espera el segundo argumento:
const add5: (b: number) => number = addCurried(5);
// Verifica en VS Code: `add5` tiene tipo `(b: number) => number`

add5(3); // 8
add5(10); // 15
```

Nota de Gabbrielli & Martini (§11.1.1): la definición `val add = fn x => (fn y => y + x)` en ML es exactamente currificación: primero se aplica `add` a `x`, obteniendo una función que espera

y. El acto de “aplicar parcialmente” un argumento NO consume el argumento completamente, sino que retorna una nueva función esperando los restantes.

```
// Curricación en pipelines
const nums = [1, 2, 3, 4, 5];
nums.map(addCurried(10)); // [11, 12, 13, 14, 15]
// Aquí addCurried(10) se evalúa a una función parcial que luego map aplica a cada número.

// Más expresivo que:
nums.map(n => add(n, 10)); // equivalente pero menos componible
```

2.7 6. Manejo de efectos y evaluación perezosa

2.7.1 6.1 Aislar los efectos

Un programa real necesita leer archivos, hacer llamadas HTTP, escribir en pantalla. La estrategia funcional es **aislar los efectos en los bordes del sistema**, manteniendo el núcleo puro:

```

      MUNDO IMPURO
[Leer input]           [Escribir output]
      ↓                   ↑

```

```

      NÚCLEO PURO
Transformaciones puras
Sin efectos laterales
Testeable en aislamiento

```

```
// Núcleo puro - fácil de testear
const calcularDescuento = (precio: number, porcentaje: number): number =>
  precio * (1 - porcentaje / 100);

// Efecto aislado en el borde (UI layer)
const mostrarDescuento = (precio: number, pct: number): void => {
  const resultado = calcularDescuento(precio, pct); // usa el núcleo puro
  console.log(`Precio final: ${resultado}`);        // efecto: I/O
};
```

2.7.2 6.2 Evaluación perezosa con generadores

En lenguajes funcionales puros como Haskell, la evaluación es **perezosa** (*lazy*) por defecto: los valores se calculan solo cuando se necesitan. En TypeScript esto se logra de forma explícita con **generadores**:

```
// Caso especial: function* no tiene sintaxis arrow - única excepción al estilo expresivo.  
// Su estado interno (n) es invisible para el exterior + la interfaz sigue siendo pura.  
function* naturales(): Generator<number> {  
  let n = 0;  
  while (true) yield n++;  
}  
  
// const arrow - consumir un generador con for-of es idiomático en TS/JS.  
const tomar = <T>(n: number, gen: Generator<T>): T[] => {  
  const res: T[] = [];  
  for (const v of gen) {  
    res.push(v);  
    if (res.length >= n) break;  
  }  
  return res;  
};  
  
tomar(5, naturales()); // [0, 1, 2, 3, 4]  
// El generador produce solo 5 valores, sin calcular los infinitos restantes
```

Caso de uso práctico: procesar streams de datos grandes sin cargar todo en memoria.

```
// Pipeline lazy para procesar grandes archivos línea a línea  
function* lineasFiltradas(lineas: string[], texto: string): Generator<string> {  
  for (const linea of lineas) {  
    if (linea.includes(texto)) yield linea;  
  }  
}
```

En **Haskell**, `[1..]` es una lista infinita completamente válida — se evalúa solo lo que se consume. En TypeScript, los generadores permiten imitar este comportamiento de forma selectiva.

2.8 7. Introducción a mónadas

2.8.1 7.1 El problema que resuelven

Las **mónadas** son un patrón de diseño para encadenar cálculos que tienen *contexto*: opcionalidad, posibilidad de error, asincronía, etc.

Sin mónadas, manejar estos contextos genera código verboso y frágil:

Código sin protección — falla al primer null:

```
const obtenerCiudad = (datos: any): string =>
  datos.usuario.direccion.ciudad.toUpperCase();
// Si datos es null, o datos.usuario es undefined, esto explota.
```

Código con verificaciones manuales — tedioso y propenso a olvidos:

```
const obtenerCiudadSeguro = (datos: any): string | null => {
  if (!datos) return null;
  if (!datos.usuario) return null;
  if (!datos.usuario.direccion) return null;
  if (!datos.usuario.direccion.ciudad) return null;
  return datos.usuario.direccion.ciudad.toUpperCase();
};
```

El problema real: - ¿Qué pasa si olvidás una verificación? → Crash en producción. - ¿Qué pasa si agregás un nuevo campo anidado? → Tenés que agregar otra verificación manual. - ¿Cómo testear cada rama de null? → Tests tedioso y propenso a falsos negativos.

La solución monádica: Encadenar las transformaciones, y si en cualquier paso obtenés null, toda la cadena se colapsa a null automáticamente. Sin verificaciones anidadas.

```
// Con mónada Option: seguro, conciso, imposible olvidar una verificación
const resultado = flatMapOpt(u => u.direccion)
  .flatMapOpt(d => d.ciudad)
  .map(c => c.toUpperCase());
// Si algún paso retorna null, el resultado es null. Fin.
```

Las mónadas transforman el “flujo de control” manual (if-null) en un patrón declarativo que el compilador puede verificar.

2.8.2 7.2 Option / Maybe — mónada de opcionalidad

```
// Tipo Option: representa un valor que puede o no existir
type Option<A> = { tag: "some"; value: A } | { tag: "none" };

const some = <A>(value: A): Option<A> => ({ tag: "some", value });
const none = <A = never>(): Option<A> => ({ tag: "none" });

// map - transforma el valor si existe
const mapOpt = <A, B>(fn: (a: A) => B) => (opt: Option<A>): Option<B> =>
  opt.tag === "some" ? some(fn(opt.value)) : none();

// flatMap (bind) - encadena operaciones que pueden fallar
const flatMapOpt = <A, B>(fn: (a: A) => Option<B>) => (opt: Option<A>): Option<B> =>
  opt.tag === "some" ? fn(opt.value) : none();

// Uso: pipeline seguro sin if-null manuales
const obtenerUsuario = (id: number): Option<{ nombre: string; edad: number }> =>
  id === 1 ? some({ nombre: "Ana", edad: 22 }) : none();

const aNombreUpper = (u: { nombre: string }): Option<string> =>
  some(u.nombre.toUpperCase());

const resultado = flatMapOpt(aNombreUpper)(obtenerUsuario(1));
// { tag: "some", value: "ANA" }

const vacio = flatMapOpt(aNombreUpper)(obtenerUsuario(99));
// { tag: "none" }
```

2.8.3 7.3 Either — mónada de error explícito

```
// Either: éxito (Right) o error (Left)
type Either<E, A> =
  | { tag: "left"; error: E }
  | { tag: "right"; value: A };

const left = <E>(error: E): Either<E, never> => ({ tag: "left", error });
```

```

const right = <A>(value: A): Either<never, A> => ({ tag: "right", value });

// flatMap para Either
const flatMapEither = <E, A, B>(
  fn: (a: A) => Either<E, B>
) => (either: Either<E, A>): Either<E, B> =>
  either.tag === "right" ? fn(either.value) : either;

// Uso: parseo seguro
const parsearNumero = (s: string): Either<string, number> => {
  const n = Number(s);
  return isNaN(n)
    ? left(`${s} no es un número válido`)
    : right(n);
};

const dividir = (divisor: number) => (n: number): Either<string, number> =>
  divisor === 0 ? left("División por cero") : right(n / divisor);

// Pipeline: parsear + dividir
const calcular = (entrada: string, divisor: number) =>
  flatMapEither(dividir(divisor))(parsearNumero(entrada));

calcular("42", 7); // { tag: "right", value: 6 }
calcular("abc", 7); // { tag: "left", error: '"abc" no es un número válido' }
calcular("42", 0); // { tag: "left", error: "División por cero" }

```

2.8.4 7.4 Promise como mónada

“Ya usás mónadas sin saberlo.”

Promise en JavaScript es una mónada: `.then()` es `flatMap`, `.catch()` maneja el contexto de error.

```

// .then() encadena transformaciones (flatMap)
fetch("/api/usuario")
  .then(res => res.json()) // si falla, los .then() siguientes se omiten
  .then(data => data.nombre)
  .catch(err => "desconocido"); // maneja el error (left)

```



```
// async/await es syntactic sugar sobre la mónada Promise
const obtenerNombre = async (): Promise<string> => {
  const res = await fetch("/api/usuario");
  const data = await res.json();
  return data.nombre;
};
```

En el **Tema 05** profundizaremos en las mónadas IO, Task y en el patrón completo de monadic error handling en TypeScript.

2.9 8. Ejemplos trabajados paso a paso

2.9.1 Ejemplo 1: Pipeline de procesamiento de encuesta

Problema: Dada una lista de respuestas de encuesta (con texto libre, algunas vacías, con errores de formato), calcular el promedio de las calificaciones numéricas válidas.

```
// Datos de entrada simulados
const respuestas = ["5", "3", "", "abc", "4", "5", " 2 ", "10"];

// Paso 1: limpiar espacios
const limpiar = (s: string): string => s.trim();

// Paso 2: parsear a número (Option para manejar fallos)
type Option<A> = { tag: "some"; value: A } | { tag: "none" };
const some = <A>(v: A): Option<A> => ({ tag: "some", value: v });
const none = (): Option<never> => ({ tag: "none" });

const parsear = (s: string): Option<number> => {
  if (s === "") return none();
  const n = Number(s);
  return isNaN(n) ? none() : some(n);
};

// Paso 3: validar rango (1-10)
const validarRango = (n: number): Option<number> =>
  n >= 1 && n <= 10 ? some(n) : none();
```

```

// Paso 4: extraer el valor si existe
const extraer = <A>(opt: Option<A>): A[] =>
  opt.tag === "some" ? [opt.value] : [];

// flatMap para Option
const flatMapOpt = <A, B>(fn: (a: A) => Option<B>) => (opt: Option<A>): Option<B> =>
  opt.tag === "some" ? fn(opt.value) : none();

// Pipeline completo
const calificacionesValidas = respuestas
  .map(limpiar)
  .map(parsear)
  .map(flatMapOpt(validarRango))
  .flatMap(extraer);
// [5, 3, 4, 5, 2]
// "10" quedó fuera del rango válido (> 10)

const promedio = calificacionesValidas.reduce((acc, n) => acc + n, 0)
  / calificacionesValidas.length;
// 3.8

```

Análisis del pipeline: - Cada paso es una función pura y testeable en aislamiento. - Los errores (cadenas vacías, no numéricas, fuera de rango) se manejan sin excepciones. - El flujo es legible de arriba hacia abajo.

2.9.2 Ejemplo 2: Mini-librería de transformaciones funcionales

Tarea: Crear una librería pequeña con `compose`, `map`, `filter`, `reduce` y usarla para resolver un problema.

```

// lib/fn.ts - mini-librería funcional

export const compose =
  <A, B, C>(f: (b: B) => C, g: (a: A) => B) =>
    (a: A): C =>
      f(g(a));

export const pipe =

```

```

<A, B, C>(g: (a: A) => B, f: (b: B) => C) =>
(a: A): C =>
  f(g(a));

export const curry =
  <A, B, C>(fn: (a: A, b: B) => C) =>
    (a: A) =>
      (b: B): C =>
        fn(a, b);

// Uso: transformar un dataset de productos

type Producto = { nombre: string; precio: number; stock: number };

const productos: readonly Producto[] = [
  { nombre: "Teclado", precio: 5000, stock: 10 },
  { nombre: "Mouse", precio: 2500, stock: 0 },
  { nombre: "Monitor", precio: 45000, stock: 3 },
  { nombre: "Webcam", precio: 8000, stock: 5 },
];

const tieneStock    = (p: Producto): boolean => p.stock > 0;
const precioConIVA   = (p: Producto): Producto => ({ ...p, precio: Math.round(p.precio * 1.21) });
const aPrecio        = (p: Producto): number => p.precio;
const sumar          = (acc: number, n: number): number => acc + n;

// Pipeline: filtrar disponibles → aplicar IVA → sumar precios
const totalConIVA = productos
  .filter(tieneStock)
  .map(precioConIVA)
  .map(aPrecio)
  .reduce(sumar, 0);

console.log(totalConIVA);
// Teclado: 6050, Monitor: 54450, Webcam: 9680 → Total: 70180

```

2.10 9. Puntos clave del tema

#	Concepto	Qué recordar
1	Modelo de cómputo	Funcional = evaluación de expresiones; imperativo = modificación de estado
2	Función pura	Determinista + sin efectos secundarios
3	Inmutabilidad	No se muta — se crea una nueva versión
4	Recursión	Reemplaza los bucles; cuidado con TCO en V8
5	HOF	<code>map</code> , <code>filter</code> , <code>reduce</code> — casos especiales de fold
6	Clausura	Función + entorno léxico capturado
7	Currificación	$f(a, b) \rightarrow f(a)(b)$ — habilita aplicación parcial
8	Composición	Piezas pequeñas $\rightarrow$ transformaciones complejas
9	Aislar efectos	Núcleo puro, bordes impuros
10	Mónadas	Patrón para encadenar cálculos con contexto (<code>Option</code> , <code>Either</code> , <code>Promise</code>)

2.11 10. Autoevaluación

2.11.1 Bloque A — Preguntas conceptuales

1. ¿Cuál es la diferencia entre evaluación por expresiones (funcional) y modificación de estado (imperativo)?
2. ¿Qué dos condiciones debe cumplir una función para ser “pura”? Da un ejemplo de función impura y explicá por qué lo es.
3. ¿Por qué en lenguajes con TCO la recursión es eficiente, y por qué hay que tener cuidado en TypeScript?
4. Explicá con tus palabras qué es una clausura y cuál es la diferencia entre ámbito léxico y ámbito dinámico.
5. ¿Qué tiene en común `Promise.then()` con el `flatMap` de una mónada?

2.11.2 Bloque B — Ejercicios de código

B1. Determiná si las siguientes funciones son puras. Justificá.

```
// a)
const multiplicar = (a: number, b: number): number => a * b;

// b)
let log: string[] = [];
const registrar = (msg: string): void => { log.push(msg); };

// c)
const aleatorio = (): number => Math.random();

// d)
const mayusculas = (s: string): string => s.toUpperCase();
```

B2. Reescribí la siguiente función imperativa en estilo funcional usando solo `map`, `filter` y `reduce`:

```
function procesar(nums: number[]): number {
  let resultado = 0;
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 0) {
      resultado += nums[i] * 2;
    }
  }
  return resultado;
}
```

B3. Implementá una función `aplanar` que tome un array de arrays y los combine en uno solo, **solo con `reduce`**, sin usar `flat()`:

```
// Entrada:  [[1, 2], [3, 4], [5]]
// Salida:   [1, 2, 3, 4, 5]
const aplanar = <T>(arr: readonly (readonly T[])[]): T[] =>
  // Tu implementación aquí
```

B4. Dado el tipo `Usuario = { nombre: string; edad: number; activo: boolean }` y un arreglo de usuarios: - Usando `filter` y `map`, obtené los nombres en mayúsculas de los usuarios activos con más de 18 años. - Implementarlo en una sola cadena de llamadas.

B5. Escribí una función currificada `entre` que devuelva `true` si un número está entre `min` (inclusive) y `max` (exclusive). Luego usala con `filter` en un arreglo.

```
const entre = (min: number) => (max: number) => (n: number): boolean =>
  // Tu implementación
```

2.12 11. Glosario

Término	Definición
-cálculo	Sistema formal de Church (1930s) para expresar cómputo mediante funciones; base teórica del paradigma funcional
Función pura	Función sin efectos secundarios y con resultado determinístico para los mismos argumentos
Efecto secundario	Cualquier modificación observable fuera del scope de la función (I/O, mutación de estado global, etc.)
Inmutabilidad	Propiedad de un valor que no puede ser modificado tras su creación; se genera un nuevo valor en su lugar
Recursión de cola	Forma de recursión donde la llamada recursiva es la última operación; permite TCO
TCO	<i>Tail Call Optimization</i> — optimización que convierte recursión de cola en iteración eficiente
HOF	<i>Higher-Order Function</i> — función que recibe o retorna funciones
Clausura	Función que captura referencias de su entorno léxico de definición
Ámbito léxico	El scope se determina por la posición en el código fuente, no por el punto de llamada
Ciudadano de primera clase	Valor que puede guardarse en variables, pasarse como argumento y retornarse como resultado
Curriificación	Transformar $f(a, b)$ en $f(a)(b)$ — secuencia de funciones de un argumento
Aplicación parcial	Fijar K argumentos de una función de N, obteniendo una función de N-K argumentos
Composición	Construir $f \circ g$ tal que $(f \circ g)(x) = f(g(x))$ — combinar funciones en nuevas funciones
Evaluación perezosa	Estrategia donde los valores se calculan solo cuando se necesitan

Término	Definición
Mónada	Patrón que permite encadenar cálculos con contexto (opcionalidad, error, asincronía), respetando las leyes de identidad y asociatividad
Option / Maybe	Mónada que representa un valor que puede o no existir; evita el null pointer
Either	Mónada que representa un resultado (Right) o un error (Left); tipo de retorno alternativo a las excepciones
fold / reduce	Función que colapsa una estructura (arreglo, árbol) en un solo valor aplicando una función acumuladora

2.13 12. Referencias

- **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms*. Springer. Cap. 11: Functional Programming Paradigm.
 - §11.1: Computing Without State — fundamentos teóricos del paradigma.
 - §11.1.1: Expressions and Functions — -cálculo y funciones de orden superior.
 - §11.1.2: Computation as Reduction — modelo de evaluación funcional.
- **Sebesta, R.** (2018). *Concepts of Programming Languages*, 12th ed. Pearson. Cap. 15: Functional Programming Languages.
 - §15.1: Introduction — historia y motivación del paradigma.
 - §15.2: Mathematical Functions — base matemática de las funciones puras.
 - §15.3: Fundamentals of Functional Programming Languages.
- **Material de cátedra.** (2025). *Introducción a la Programación Funcional con Kotlin*. UNTDF — Sede Ushuaia. (*Adaptado a TypeScript para el cursado 2026.*)
- **Schmidt, D. C. & Runfola, D.** (2025). *Liberating Logic in the Age of AI: Going Beyond Programming with Computational Thinking*. William & Mary. (*Contexto del paradigma funcional en la era de IA.*)

Próxima lectura: Guía de estudio del Tema 04 — Aspectos Avanzados de Programación Funcional.

Esta guía es el insumo base para el TP del Tema 03 (ver `tp.md`).